

MIT OPEN CREDIT MANAGEMENT SYSTEM

Rule-Driven Academic Enrollment, Result & Financial Management Platform

SE801 / Software Project Lab 3

Midterm Technical Report & Concise SRS

Submitted by: Md. Jihad Hossain

Roll: BSSE 1413

Supervised by: Dr. Md. Nurul Ahad Tawhid

Institution: Institute of Information Technology, University of Dhaka

Presentation Date: 02 August 2026

Table of Contents

1. Project Overview	4
1.1 Project Title	4
1.2 Problem Statement	4
1.3 Objectives	4
1.4 Scope	5
1.5 Deliverables	5
2. Requirements Analysis	6
2.1 Functional Requirements	6
2.2 Non-functional Requirements	7
2.3 Stakeholders	7
2.4 Usage Scenarios	7
3. System Modeling	8
3.1 Use Case Diagrams & Descriptions	8
3.2 Activity Diagrams	15
3.3 Data Flow Diagrams	20
4. Data & Information Modeling	23
4.1 Data Objects & Relationships	23
4.2 Entity Relationship Diagram	24
4.3 Database Schema Summary	25
5. Software Design	26
5.1 Component Diagram	26
5.2 UI-to-Database Interaction	27
6. Software Architecture & Traceability	28
7. Preliminary Test Plan	29
8. Implementation Progress & Challenges	29
9. Timeline & Demonstration Plan	30

List of Figures

Figure 1: Use case diagram — Level 0 system overview.....	87. Preliminary Test Plan 29
Figure 2: Use case diagram — Level 1 full system.....	98. Implementation Progress & Challenges 29
Figure 3: Authentication & Role Access.....	109. Timeline & Demonstration Plan 30
Figure 4: Student Course Registration.....	11
Figure 5: Student Academic Progress & Finance.....	12
Figure 6: Teacher Marks Management.....	13
Figure 7: Admin Academic & Financial Administration.....	14
Figure 8: Authentication activity.....	15
Figure 9: Course registration activity.....	16
Figure 10: Teacher marks activity.....	17
Figure 11: Result calculation and publication activity.....	18
Figure 12: Fee and dues activity.....	19
Figure 13: DFD Level 0 — system context.....	20
Figure 14: DFD Level 1 — process decomposition.....	21
Figure 15: DFD Level 2 — semester registration.....	22
Figure 16: Logical ER diagram.....	24
Figure 17: Code-aligned component architecture.....	26
Figure 18: UI click-to-database flow.....	27
Figure 19: Layered runtime and deployment view.....	28
Figure 20: Timeline and presentation milestone.....	30

1. Project Overview

1.1 Project Title

MIT Open Credit Management System — a secure, role-based academic platform for managing open-credit enrollment, marks, results, CGPA and student financial records.

1.2 Problem Statement

The open-credit process combines semester registration, mandatory and optional courses, limited seats, teacher assignment, marks submission, result processing and financial penalties. When these activities are managed through disconnected forms or spreadsheets, credit totals may be invalid, seats may be allocated inconsistently, marks and CGPA may be calculated incorrectly, and academic or financial records may become difficult to verify. Students need a clear and trustworthy registration and progress view; Teachers need controlled access to assigned courses and rosters; and Administrators need consistent academic, result and finance management without bypassing institutional rules.

1.3 Objectives

- **O1 — Secure Access:** Authenticate Student, Teacher and Admin users with JWT and enforce role-based access at both frontend and backend.
- **O2 — Credit Control:** Support the approved requirement of exactly 12 credits per semester and 36 credits for the programme.
- **O3 — Seat Control:** Enforce a maximum of 40 seats for optional courses and protect the final seat against concurrent requests.
- **O4 — Academic Results:** Allow Teachers to enter marks and enable grade, GPA and CGPA calculation with an explicit publication workflow.
- **O5 — Finance:** Calculate retake and semester-gap fees and maintain transparent charge, payment and dues records.
- **O6 — Usability & Maintainability:** Provide clear role dashboards using Angular, Spring Boot and PostgreSQL with traceable requirements and testable rules.

1.4 Scope

In scope:

- JWT authentication and Student/Teacher/Admin RBAC.
- Student dashboard, profile, course catalogue, enrollment history, results, CGPA and dues.
- Teacher dashboard, assigned courses, enrolled student roster and validated midterm/final mark entry.
- Admin management of students, courses, semesters, fees and system statistics.
- Academic rules for credit validation, seat availability, grade calculation and CGPA.
- Angular 18 frontend, Spring Boot 3.3 / Java 21 backend, PostgreSQL 16 and Flyway V1–V7.

Out of scope for the midterm baseline:

- Mobile application, AI recommendation or chatbot features.
- Microservices, Redis, Neo4j and other speculative infrastructure.
- Full online payment-gateway settlement and unrelated university ERP modules.

1.5 Deliverables

#	Deliverable	Description
1	Angular Frontend	Role-based Student, Teacher and Admin screens with routing, forms, API services and dashboard views.
2	Spring Boot Backend	JWT security, REST controllers, service-layer validation and JPA persistence.
3	PostgreSQL Database	Flyway V1–V7 schema for users, courses, semesters, enrollments, grades and fees.
4	System Models	Use case, activity, DFD, ERD, component and deployment models.
5	Test & Presentation Package	Preliminary test plan, progress analysis, Gantt chart and demonstration sequence.

2. Requirements Analysis

2.1 Functional Requirements

ID	Functional Requirement	Current Status
FR-01	Authenticate valid users and issue JWT access based on Student, Teacher or Admin role.	Implemented
FR-02	Show role-specific dashboards and prevent cross-role access.	Implemented
FR-03	Display courses, credit values, seats and active semester information.	Implemented with limitation
FR-04	Allow a Student to create and remove own course enrollments.	Implemented with limitation
FR-05	Confirm only an exactly 12-credit semester registration; reject partial confirmation.	Target correction
FR-06	Prevent duplicate enrollment and reject a full optional course above 40 seats.	Implemented with limitation
FR-07	Allow a Teacher to view assigned courses and enrolled Students.	Implemented
FR-08	Validate and save midterm/final marks for authorized courses only.	Implemented
FR-09	Calculate total marks, grade point and CGPA accurately.	Implemented with limitation
FR-10	Publish results before they become visible to Students.	Target correction
FR-11	Generate retake and semester-gap fees and show outstanding dues.	Implemented with limitation
FR-12	Record and verify payments against outstanding charges.	Target correction
FR-13	Allow Admin to manage students, courses, semesters and fees.	Implemented with limitation
FR-14	Provide statistics, monitoring and traceable validation messages.	Implemented with limitation

2.2 Non-functional Requirements

Category	Requirement
Security	BCrypt password hashing, signed JWT, backend RBAC, ownership checks and non-revealing authentication errors.
Integrity	Enrollment, seat, grade and fee updates must avoid partial or duplicated records.
Performance	Normal pages should respond within approximately 2 seconds under presentation-scale load.
Usability	Readable responsive pages, clear validation messages, consistent navigation and no text overlap.
Maintainability	Angular components/services and Spring controller/service/repository responsibilities remain separated.
Deployability	A clean PostgreSQL database must migrate through Flyway V1-V7 and the Angular/Maven builds must succeed.

2.3 Stakeholders

Stakeholder	Main Need
Student	Secure registration, results, CGPA, fee and due visibility.
Teacher	Assigned-course roster and reliable marks entry.
Admin	Master-data management, monitoring, results and finance control.
Academic Authority	Correct enforcement of credit, seat and grade policies.
Supervisor / Evaluator	Clear, correct and consistent modelling with honest progress evidence.

2.4 Usage Scenarios

- **Student Registration:** Student signs in, opens available courses, selects a course or semester package, receives validation and sees the saved enrollment.
- **Teacher Marks:** Teacher opens an assigned course, loads the roster, enters valid marks and saves them for grade processing.
- **Admin Control:** Admin manages academic data, monitors seats and fees, and reviews result or payment status.

3. System Modeling

3.1 Use Case Diagrams and Descriptions

The use case model progresses from a high-level system view to the concrete responsibilities of Student, Teacher and Admin. The diagrams describe the required system behaviour; the implementation-progress section separately identifies current limitations.

Level 0 — System Overview

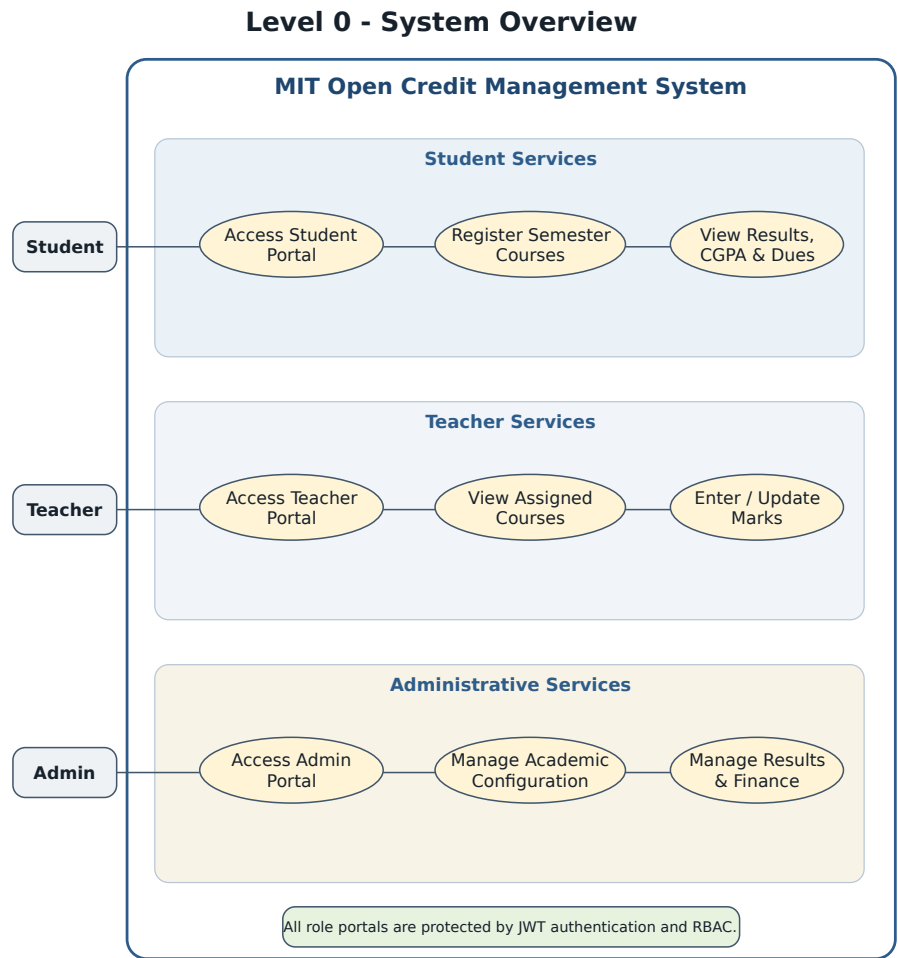


Figure 1: Use case diagram — Level 0 system overview

Students register courses and view results, CGPA and dues. Teachers manage marks and course results. Administrators manage academic and financial data. All actors enter through protected role access.

Level 1 — Full System (Actor-Based)

Level 1 - Full System (Actor-Based)

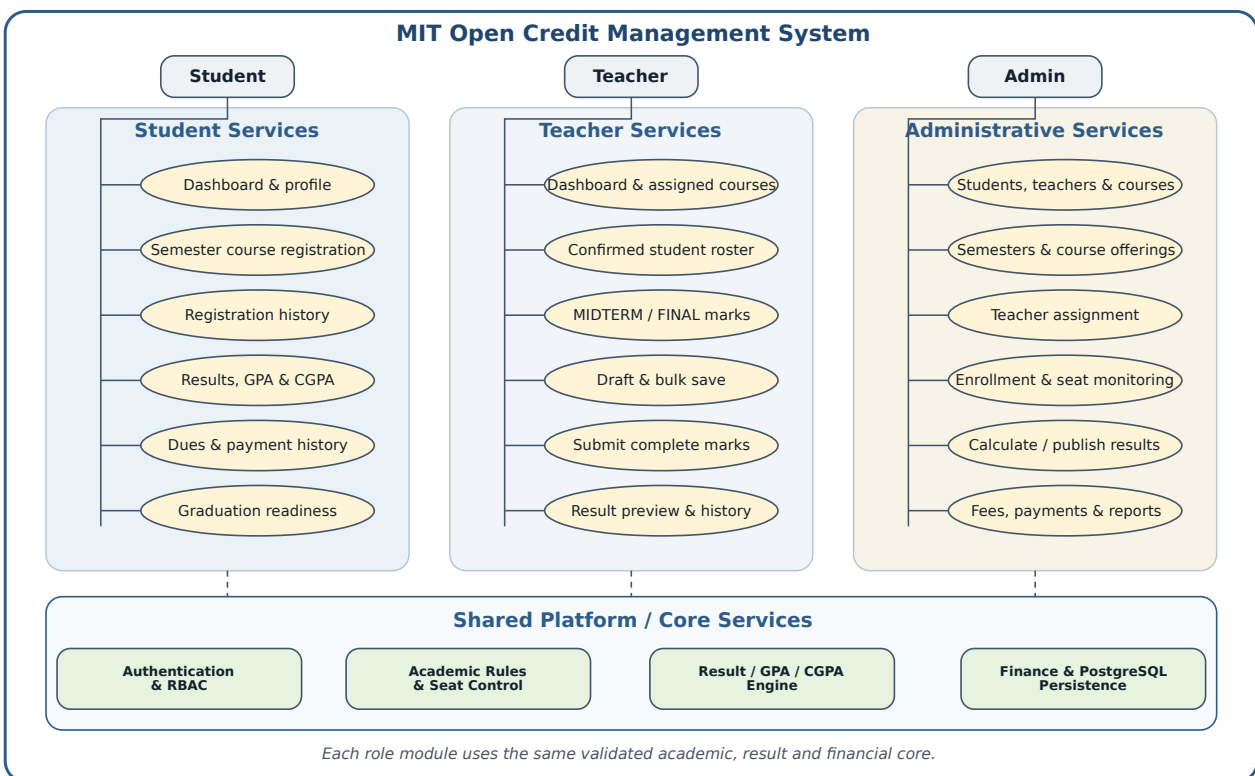


Figure 2: Use case diagram — Level 1 full system

The full-system diagram now separates Student, Academic and Administrative responsibilities into clearer clusters so the actor interactions remain readable during presentation and discussion.

Level 1.1 — Authentication & Role Access

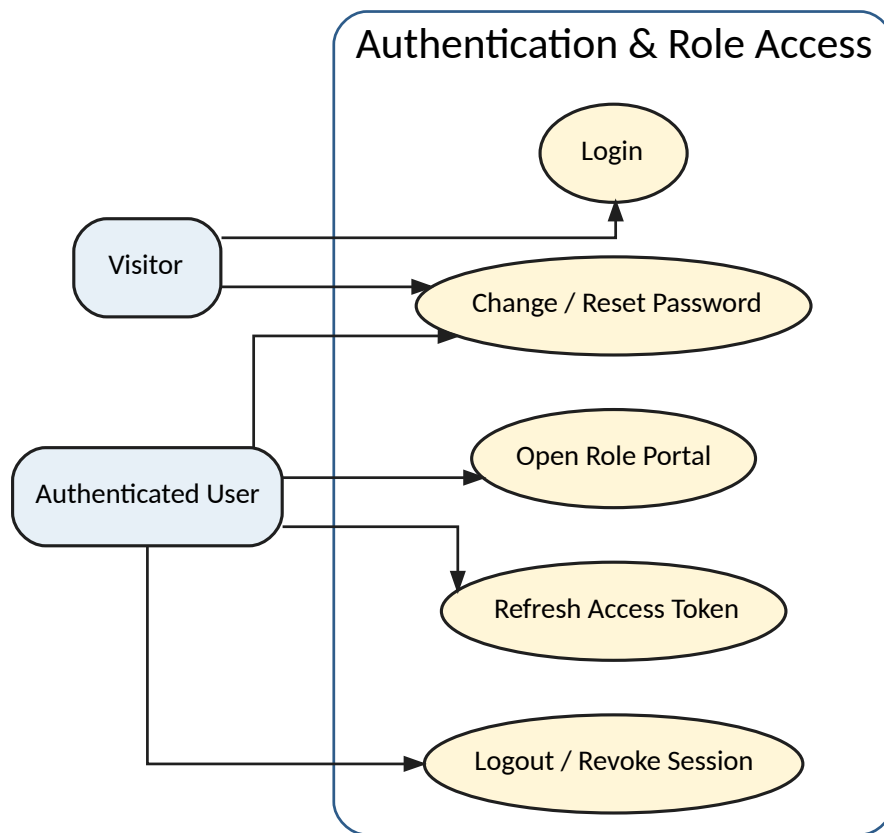


Figure 3: Use case diagram — Authentication & Role Access

Use Case Name: Authentication & Role Access

Primary Actors: Visitor, Authenticated User

Secondary Actors: —

Action - Reply

- **Action:** User submits email and password.
- **Reply:** System validates credentials, account status and role, then issues a signed JWT.
- **Action:** User opens a protected route.
- **Reply:** Frontend guard and backend security verify the required Student, Teacher or Admin role.
- **Action:** User logs out or the token expires.
- **Reply:** System ends or rejects the session and redirects the browser to login.

Level 1.2 — Student Course Registration

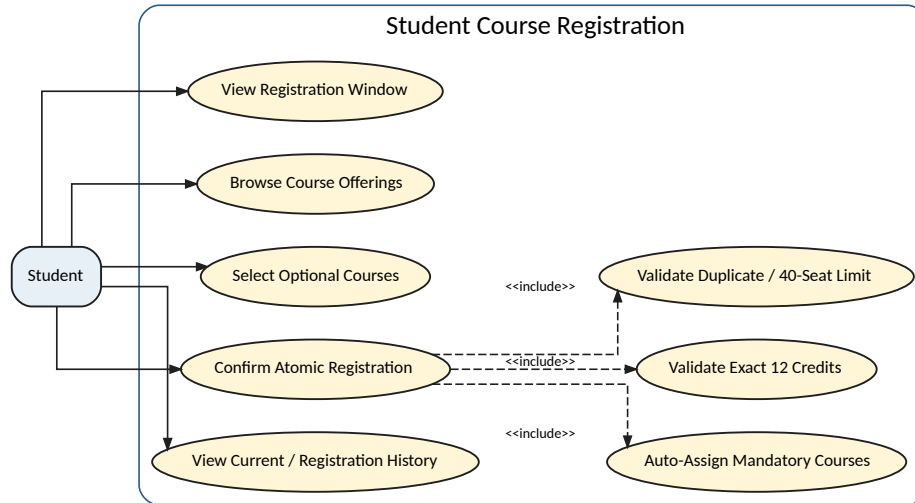


Figure 4: Use case diagram — Student Course Registration

Use Case Name: Student Course Registration

Primary Actors: Student

Secondary Actors: —

Action - Reply

- **Action:** Student opens the registration page and reviews mandatory and optional courses.
- **Reply:** System returns course, credit and seat information for the active semester.
- **Action:** Student submits a semester course package.
- **Reply:** System checks semester, duplicates, exact 12 credits and optional-course seats before saving.
- **Action:** Any validation fails.
- **Reply:** System rejects the full request with a clear reason and keeps the stored state unchanged.

Implementation note: the current project still registers courses individually and validates a maximum of 12 credits; exact-12 atomic registration is the highest-priority correction.

Level 1.3 — Student Academic Progress & Finance

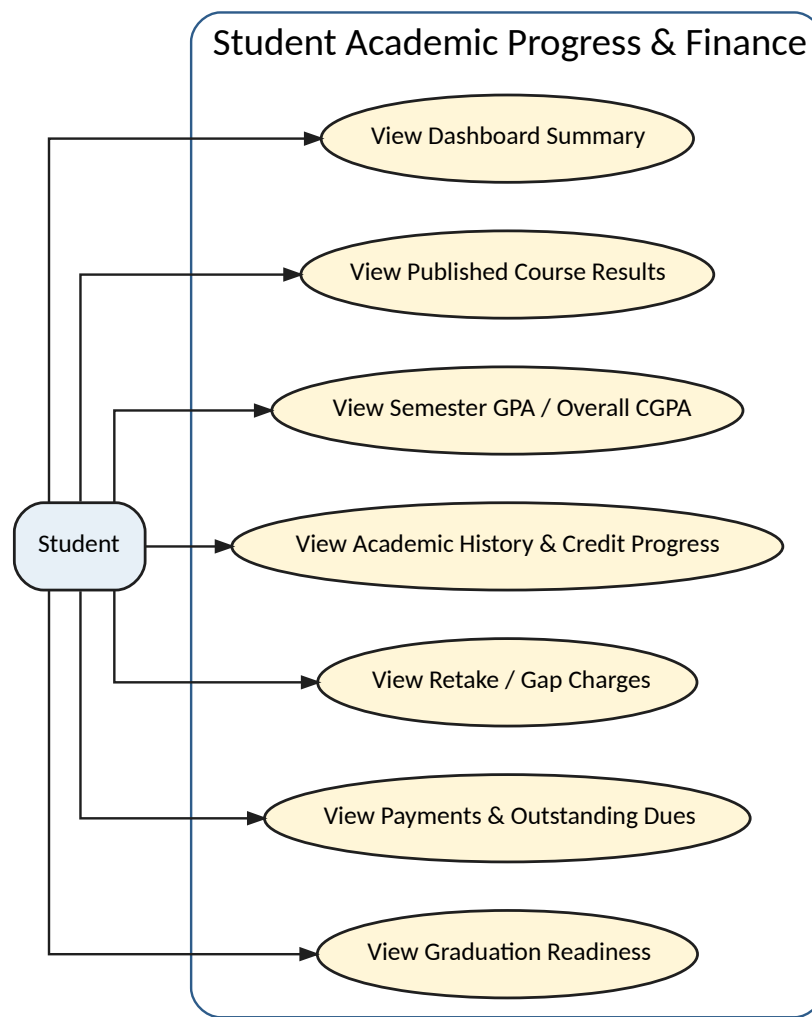


Figure 5: Use case diagram — Student Academic Progress & Finance

Use Case Name: Student Academic Progress & Finance

Primary Actors: Student

Secondary Actors: —

Action - Reply

- **Action:** Student opens the dashboard, results or dues page.
- **Reply:** System loads only the authenticated Student's enrollment, grade, CGPA and fee records.
- **Action:** Student selects a semester or course result.
- **Reply:** System shows marks, total, grade point and the CGPA contribution available in the current data.
- **Action:** Student opens financial history.
- **Reply:** System shows generated fees and outstanding dues; verified payment allocation remains a target workflow.

Level 1.4 — Teacher Marks Management

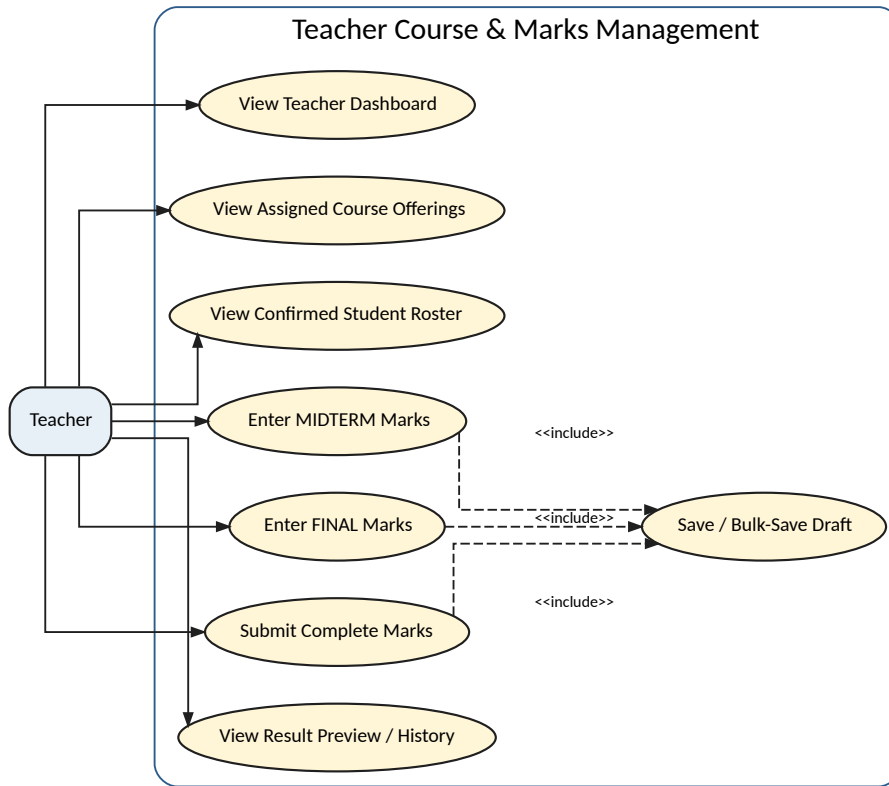


Figure 6: Use case diagram — Teacher Marks Management

Use Case Name: Teacher Marks Management

Primary Actors: Teacher

Secondary Actors: Student Roster

Action - Reply

- **Action:** Teacher opens an assigned course.
- **Reply:** System verifies course ownership and returns enrolled Students.
- **Action:** Teacher enters midterm and final marks.
- **Reply:** System validates the allowed range and saves a grade record for the correct enrollment.
- **Action:** Teacher attempts to access an unassigned course.
- **Reply:** System rejects the request; a formal submit/publish lifecycle remains a planned refinement.

Level 1.5 — Admin Academic & Financial Administration

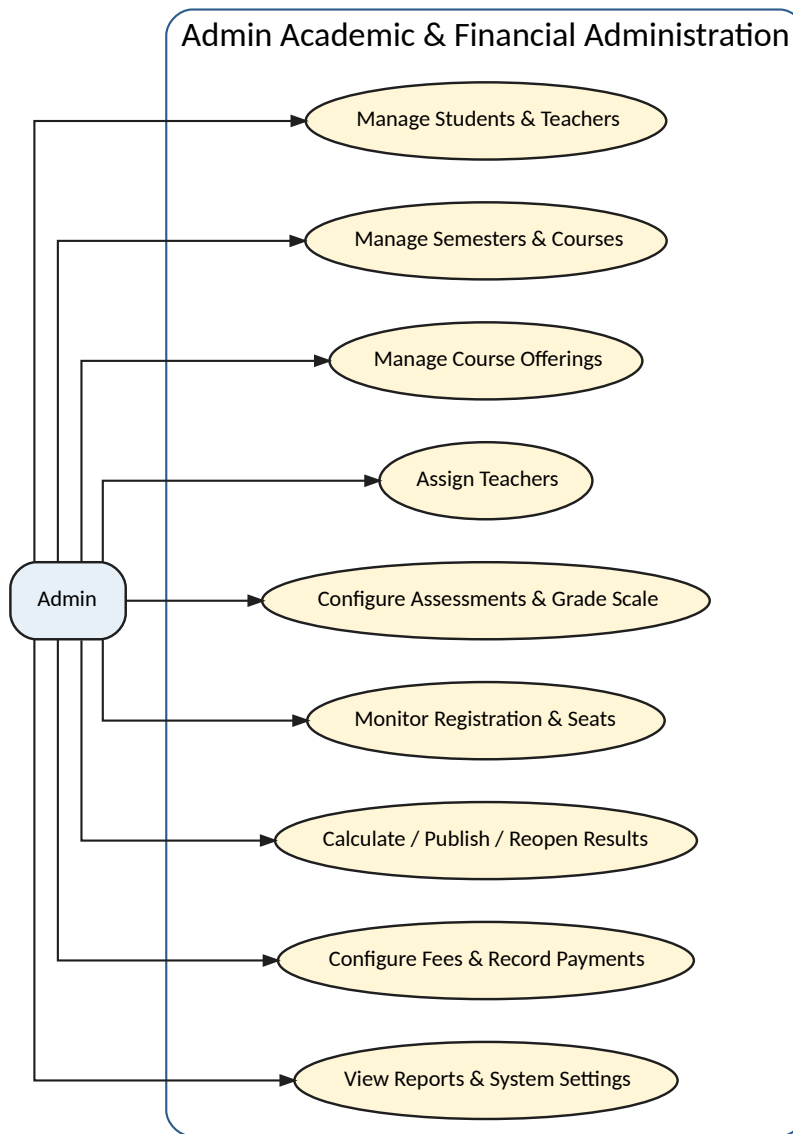


Figure 7: Use case diagram — Admin Academic & Financial Administration

Use Case Name: Admin Academic & Financial Administration

Primary Actors: Admin

Secondary Actors: Student, Teacher

Action - Reply

- **Action:** Admin creates or updates students, courses, semesters or fees.
- **Reply:** System validates the data and stores the authorized change.
- **Action:** Admin opens statistics or monitoring.
- **Reply:** System aggregates current academic and financial records.
- **Action:** Admin publishes results or records a payment.
- **Reply:** The required target workflow validates state, preserves auditability and prevents invalid balance changes.

3.2 Activity Diagrams

Activity diagrams show the dynamic flow of the most important workflows. Decision paths are explicit so the presentation can explain both successful and rejected cases.

3.2.1 Authentication & Role Routing

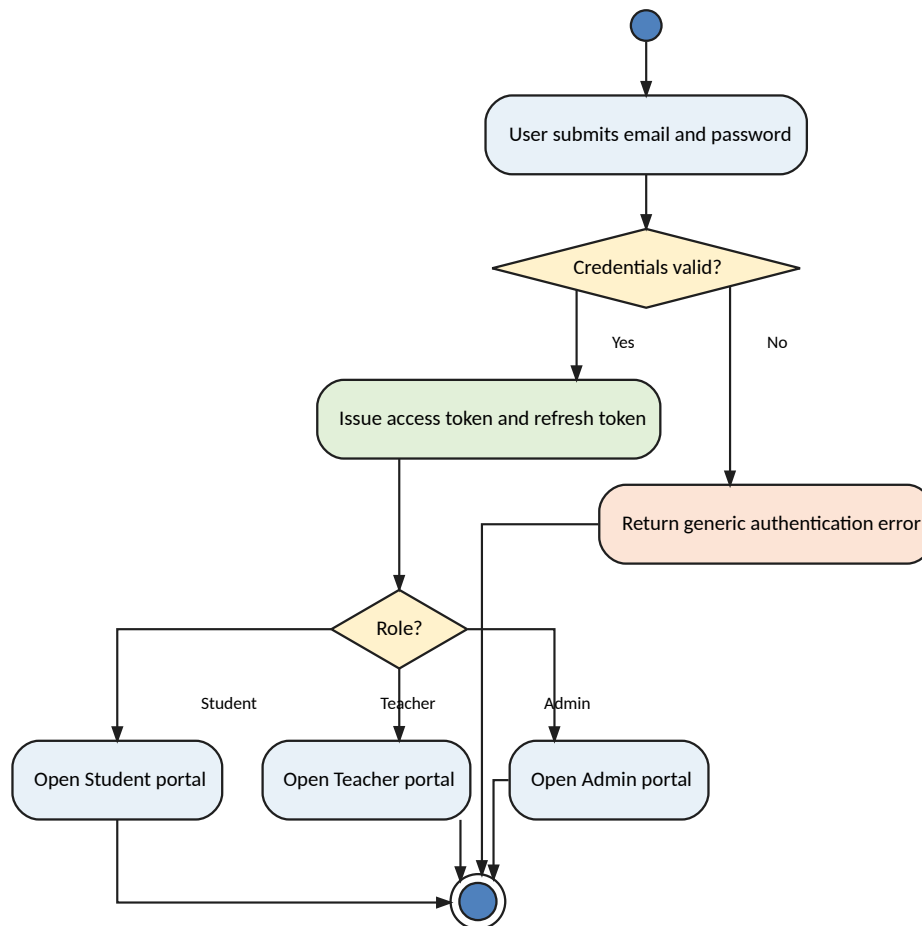


Figure 8: Activity diagram — Authentication & Role Routing

The system validates credentials and account status before routing the user to the correct dashboard. Invalid credentials or role mismatches terminate the attempt without exposing protected data.

3.2.2 Student Course Registration

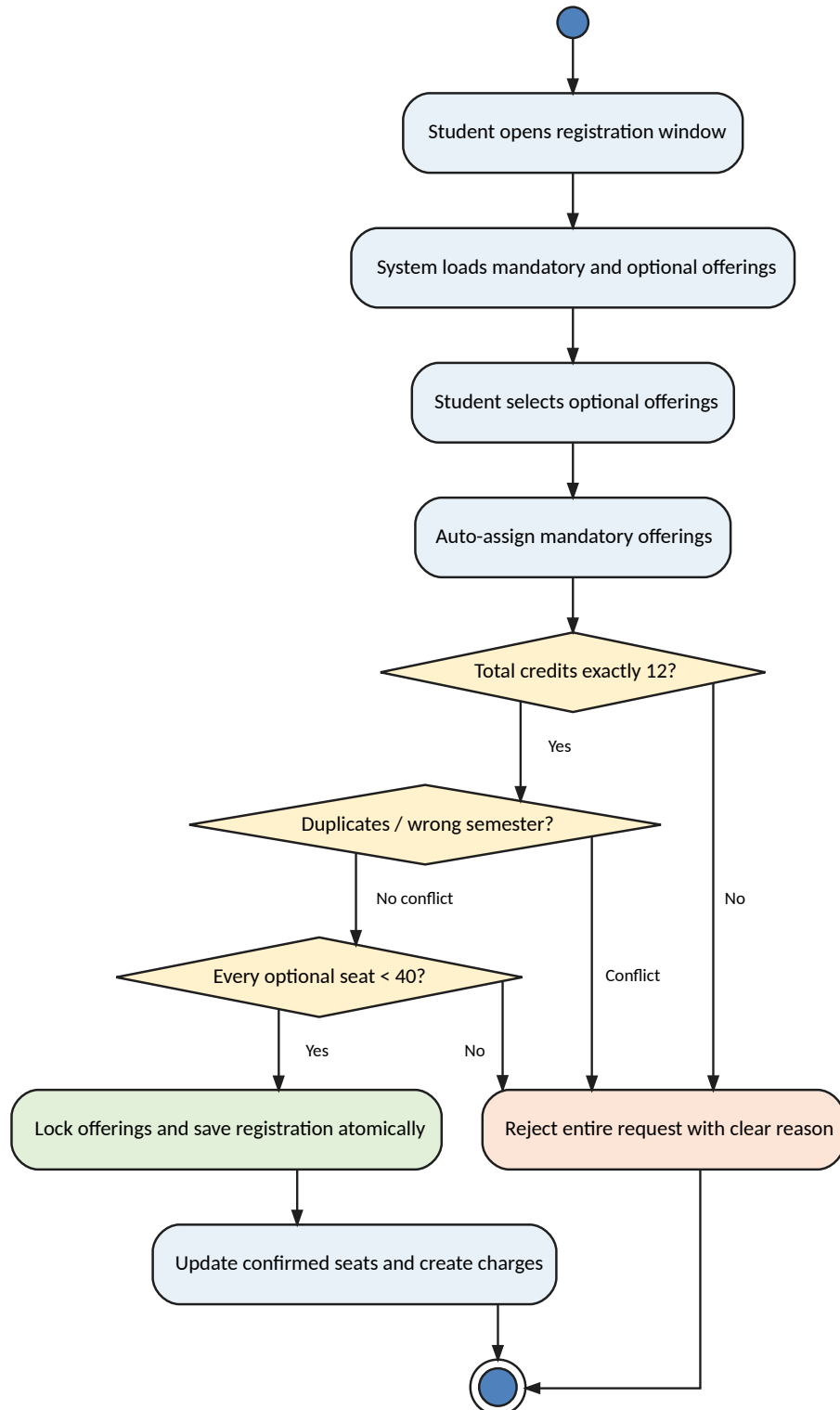


Figure 9: Activity diagram — Student Course Registration

This is the required corrected flow: build the complete package, validate exact credits, duplicates and seats, then persist all records as one transaction. The current course-by-course implementation is a partial predecessor of this flow.

3.2.3 Teacher Marks Entry

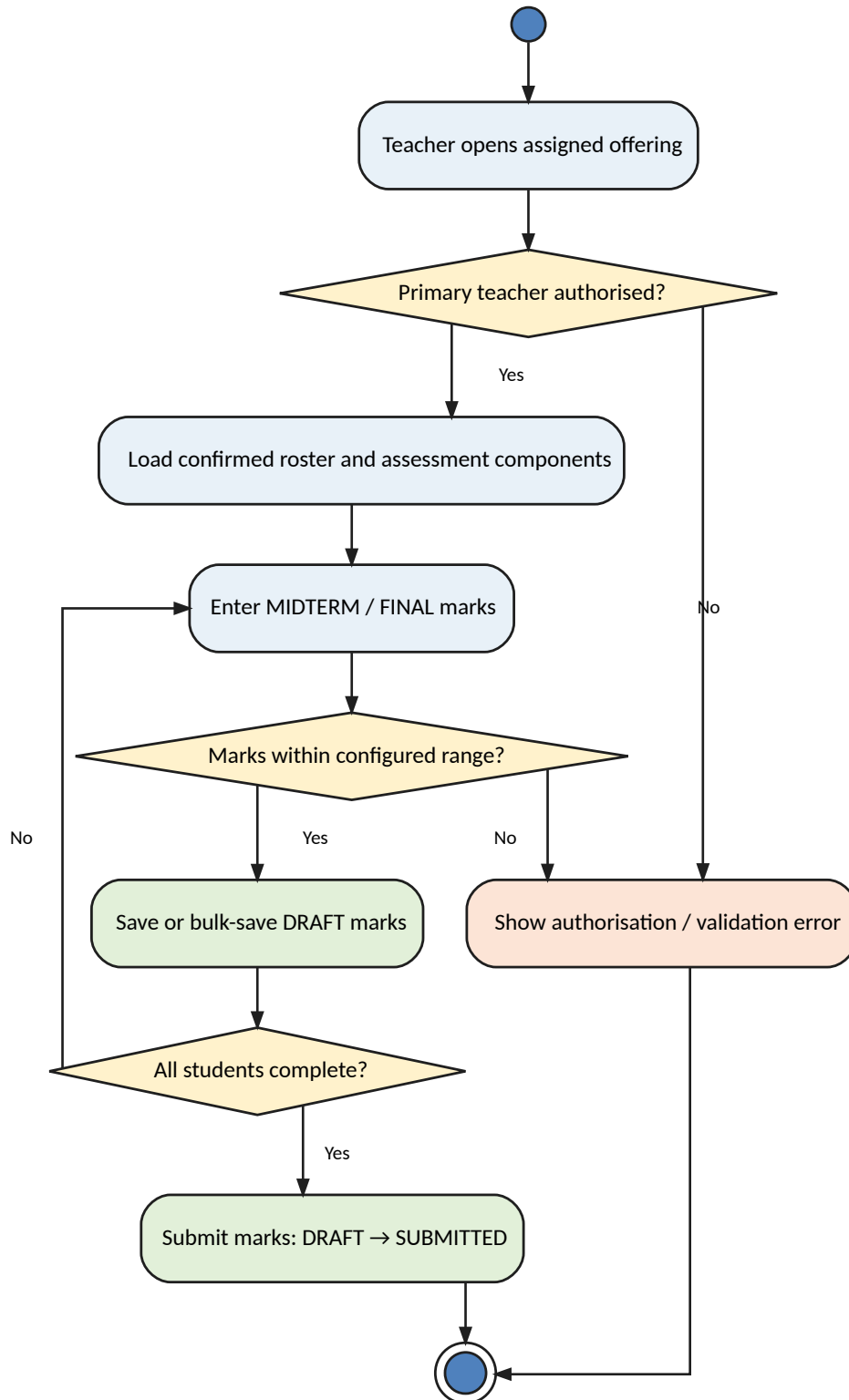


Figure 10: Activity diagram — Teacher Marks Entry

Teacher authorization, roster ownership and marks-range validation protect the grade-entry process. Draft/submit states are the recommended completion model.

3.2.4 Result Calculation & Publication

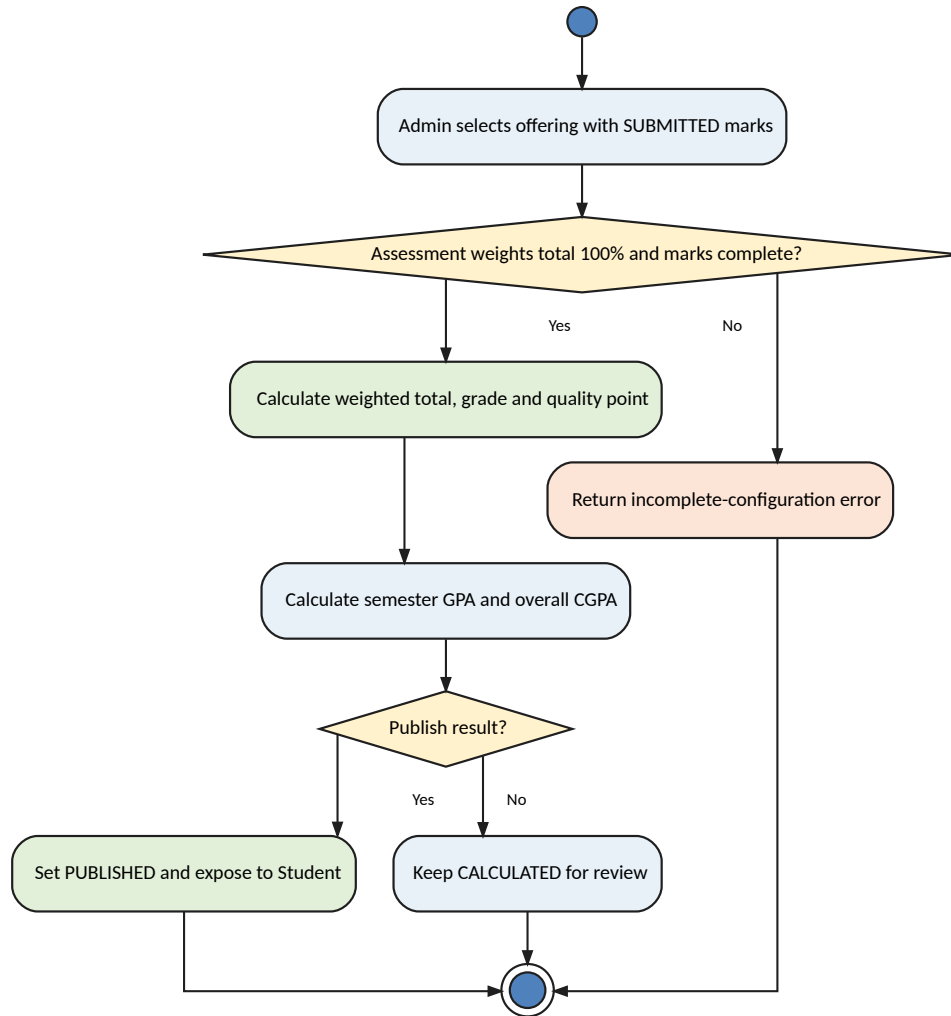


Figure 11: Activity diagram — Result Calculation & Publication

The intended lifecycle separates mark entry from calculation and publication. Students should see only published results; the current build calculates grades and CGPA but lacks this explicit lifecycle.

3.2.5 Fee, Payment & Dues Update

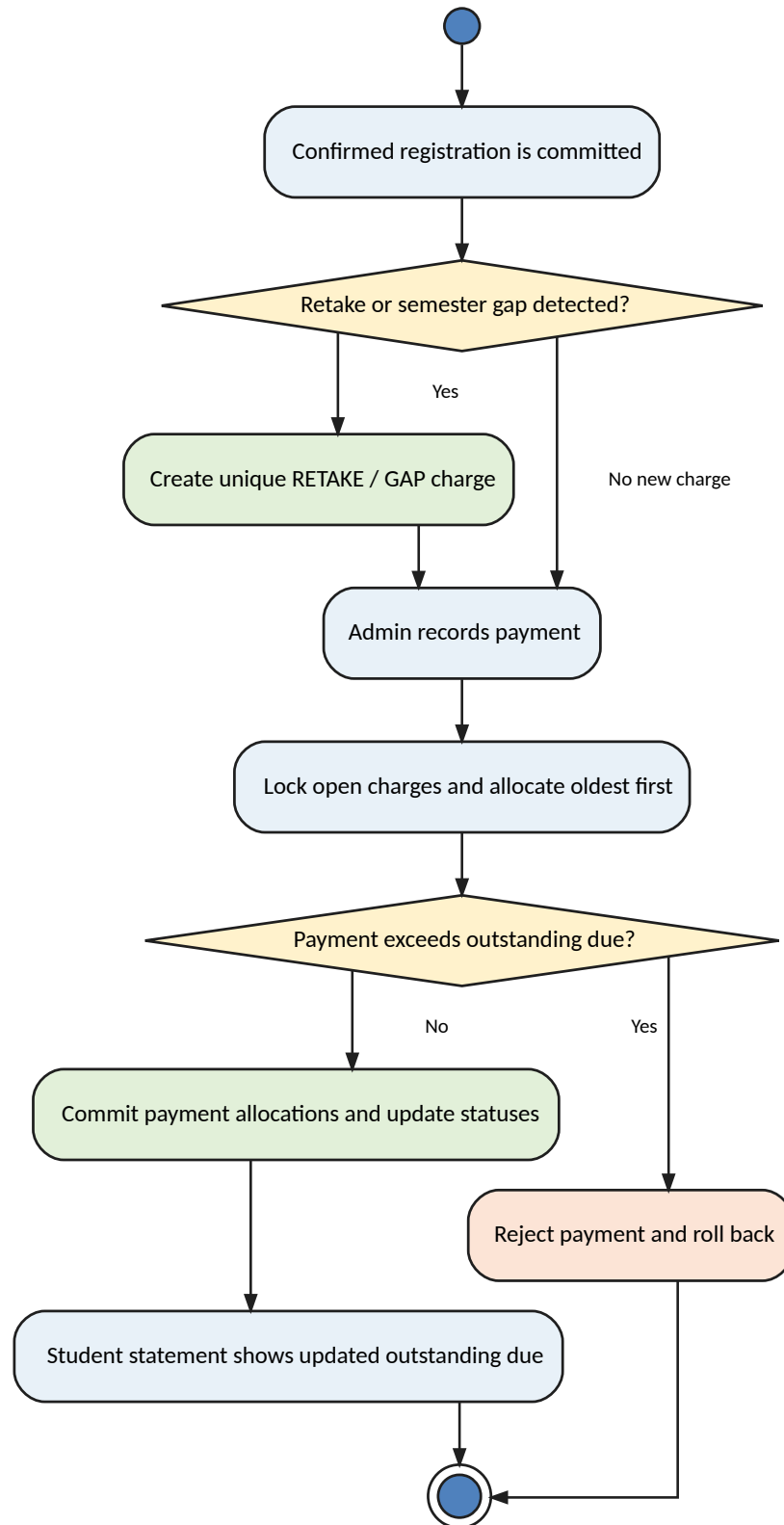


Figure 12: Activity diagram — Fee, Payment & Dues Update

Retake and semester-gap conditions should be derived from academic history. Payments should be verified and allocated to charges rather than represented only as a status change.

3.3 Data Flow Diagrams

3.3.1 DFD Level 0 — System Context

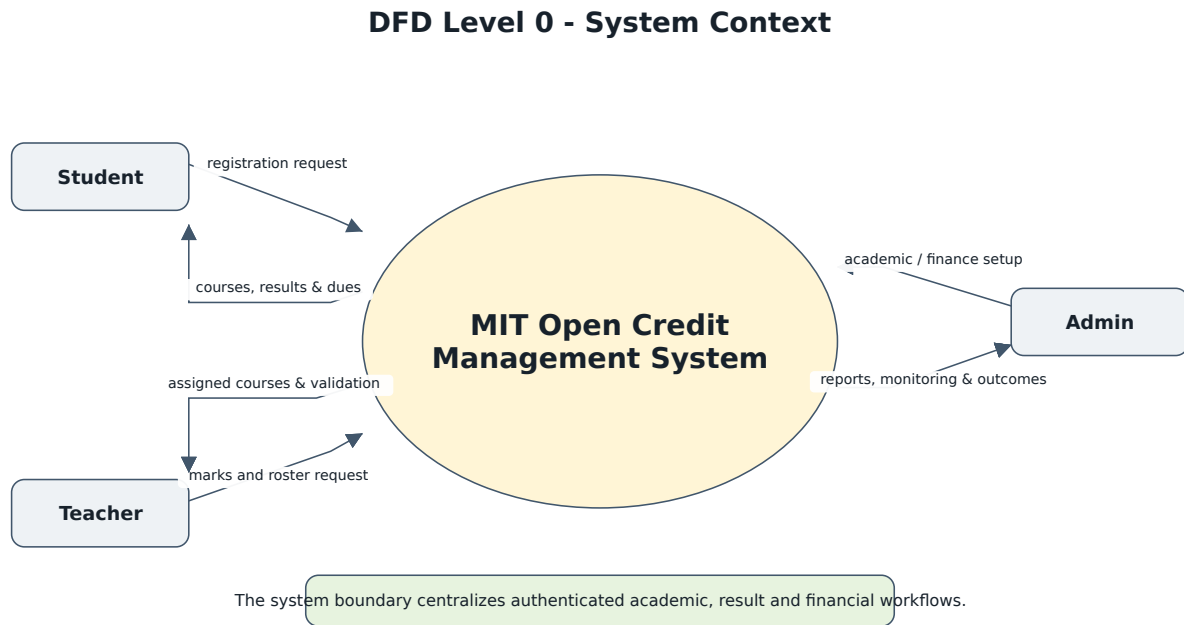


Figure 13: DFD Level 0 — system context

The context diagram treats MIT OCMS as one process and shows the academic and financial information exchanged by each external actor.

3.3.2 DFD Level 1 — Major Processes

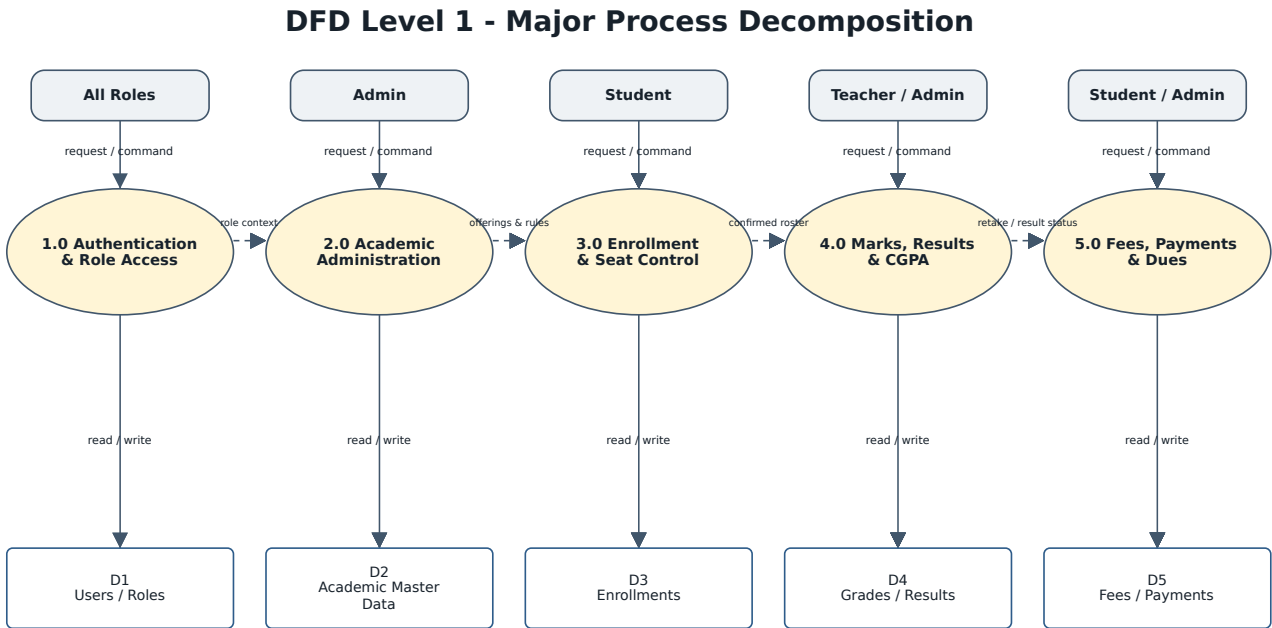
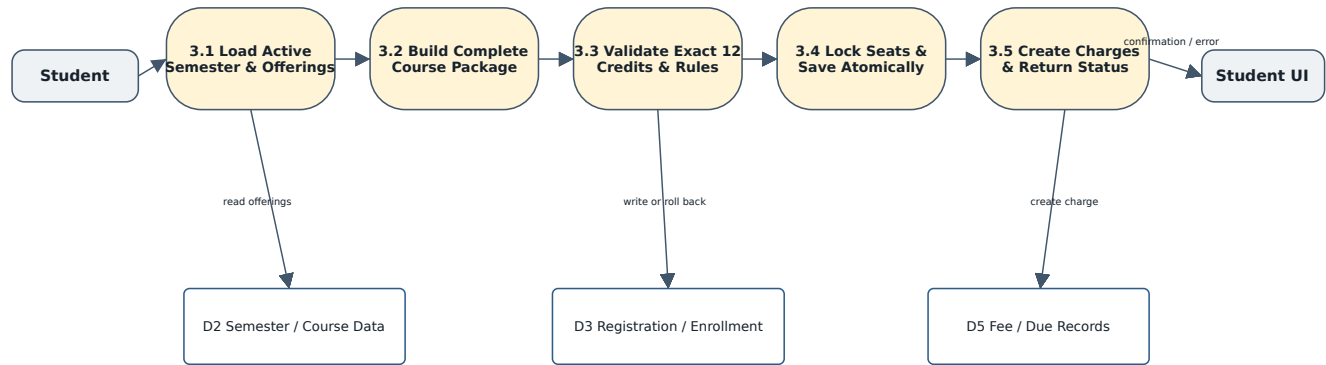


Figure 14: DFD Level 1 — major process decomposition

Level 1 now separates authentication, academic administration, enrollment, marks/results and finance in a more readable layout while preserving the same backend data responsibilities.

3.3.3 DFD Level 2 — Student Semester Registration

DFD Level 2 - Student Semester Registration



Target correction: the exact-12-credit package succeeds or fails as one transaction.

Figure 15: DFD Level 2 — semester registration

The registration decomposition now shows a clearer left-to-right flow: load semester data, build the full package, validate exact 12 credits and seat rules, then persist or reject atomically.

4. Data & Information Modeling

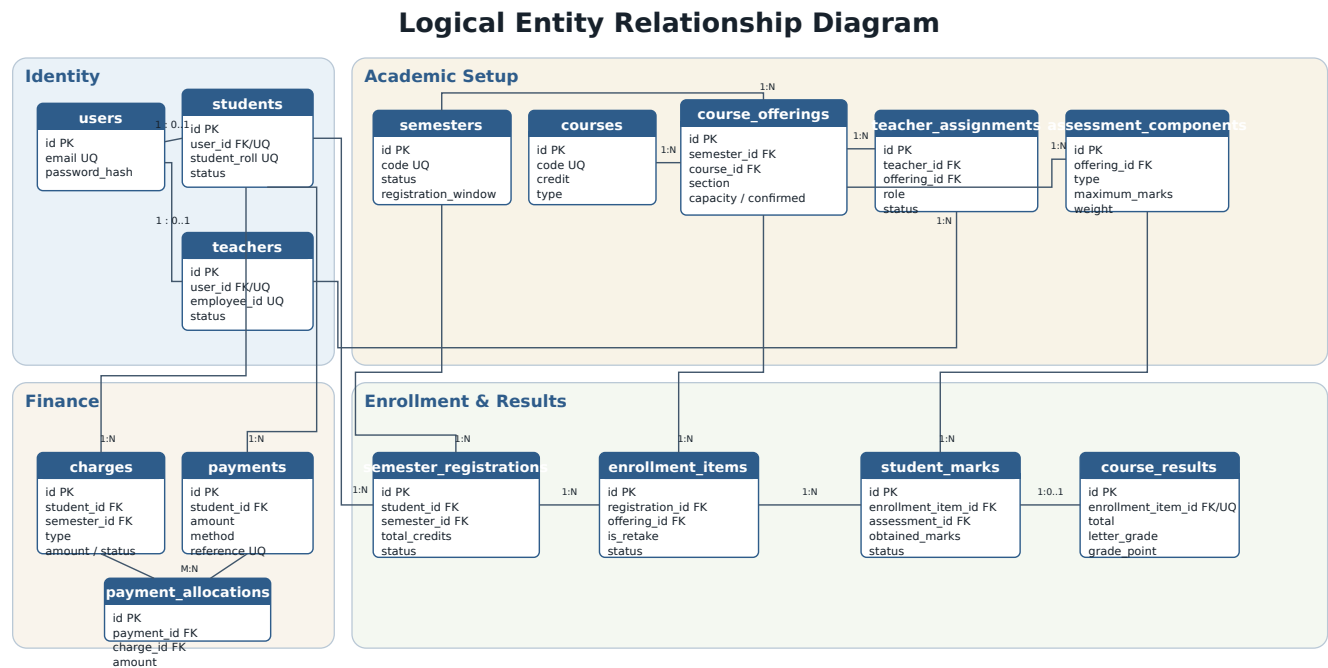
4.1 Data Objects and Relationships

The data model is derived from the recurring problem-space nouns in requirements and usage scenarios. Interface terms such as page, button, token and API are solution concepts; the objects below carry persistent academic or financial meaning.

Data Object	Purpose	Main Relationships
User	Authentication identity and role.	One user may own one Student or Teacher profile.
Student / Teacher	Role-specific identity and status.	Student owns enrollments, grades and fees; Teacher owns assigned courses.
Semester	Academic term and registration context.	Contains enrollments and should own course offerings.
Course	Course code, title, credit, type and capacity.	Referenced by enrollments and grades; current seat/teacher data are global.
Enrollment	Student–Course–Semester association.	Links Student, Course, Semester and Grade.
Grade	Midterm, final, total, letter grade and point.	Belongs to one Enrollment.
Fee / Payment	Retake, gap and other financial records.	Belongs to a Student and may reference a Semester or Enrollment.
CourseOffering (required)	Semester-specific course section, seat count and Teacher.	Required to correct global seat/teacher limitations.

Relationship summary: A Student enrolls in Courses during a Semester; a Teacher handles an assigned course; each Enrollment may have one Grade; a Student may have multiple Fees and Payments. The corrected model inserts CourseOffering between Course/Semester and Enrollment.

4.2 Entity Relationship Diagram



Key attributes are shown for presentation; detailed constraints remain in the database schema specification.

Figure 16: Logical ER diagram

The ERD shows the required logical next state, including semester-specific CourseOffering, TeacherAssignment, SemesterRegistration and PaymentAllocation. The current project contains a simplified subset centred on User, Course, Semester, Enrollment, Grade and Fee.

4.3 Database Schema Summary

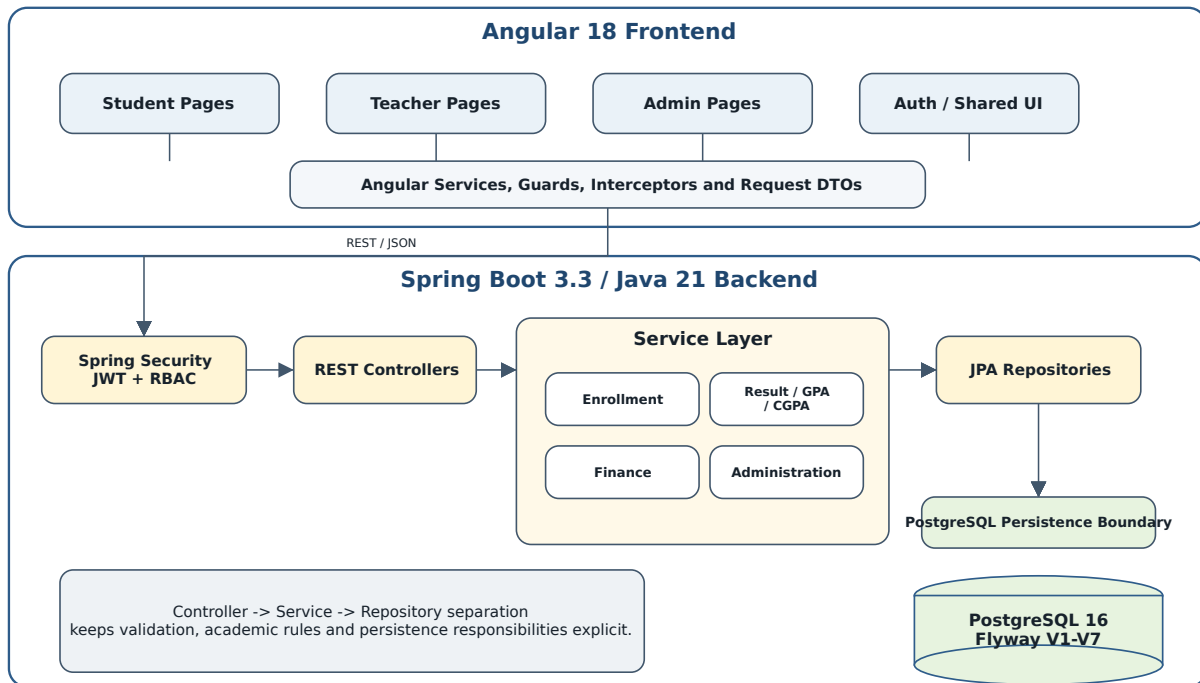
Table / Entity	Key Fields	Constraint or Presentation Note
users	id, email, password_hash, role, active	Unique email; BCrypt password; privileged role creation must be restricted.
students / teachers	id, user_id, roll/employee_id	One profile per user; unique roll or employee ID.
courses	id, code, title, credit, type, capacity	Current capacity and Teacher assignment are global; move to offering.
semesters	id, name/year/term, active	Registration-window uniqueness requires refinement.
enrollments	id, student_id, course_id, semester_id, status	Unique student + course + semester; current flow is course-by-course.
grades	id, enrollment_id, midterm, final, total, point	One grade per enrollment; publication state is absent.
fees / payments	id, student_id, type, amount, status	Current payment verification/allocation and gap automation are incomplete.
course_offerings (required)	semester_id, course_id, section, teacher_id, capacity, seats	Required for exact semester registration and concurrency-safe seats.
semester_registrations (required)	student_id, semester_id, total_credits, status	Required aggregate for exactly 12 credits and atomic commit.

Database authority: Backend services and PostgreSQL constraints—not the browser—must make the final decision for credits, duplicates, seats, grades and balances.

5. Software Design

5.1 Component Diagram

Current Code-Aligned Component Architecture

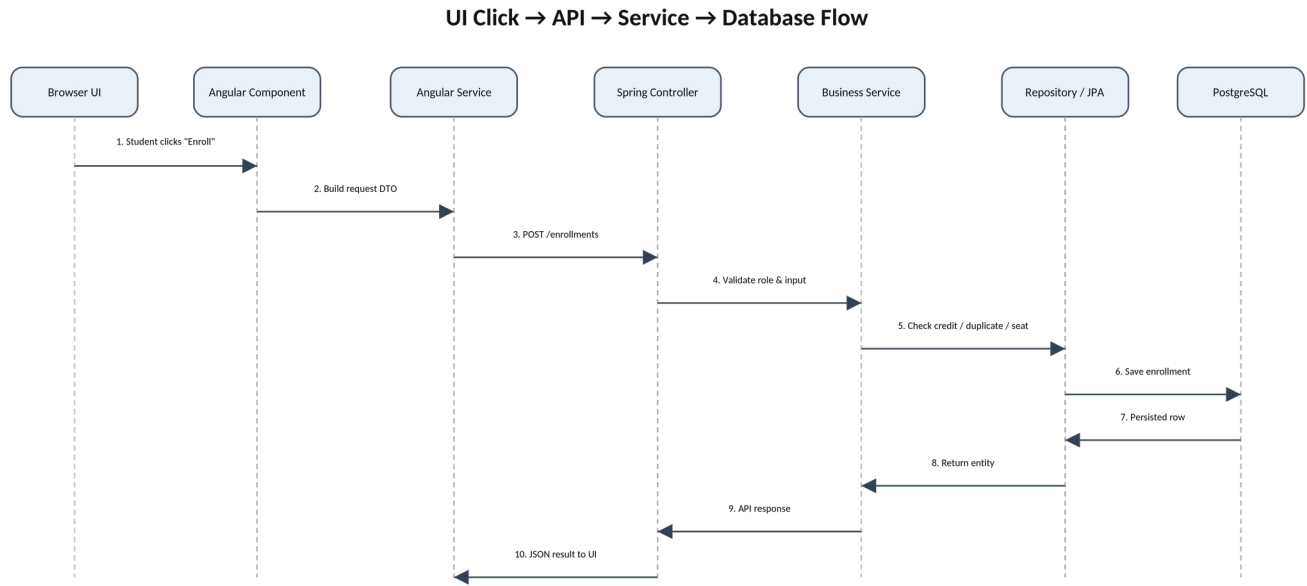


Single Spring Boot application; no unnecessary microservice or database complexity.

Figure 17: Code-aligned component architecture

The component diagram highlights the actual Angular → Spring Security/Controller → Service → Repository → PostgreSQL path with larger, clearer boundaries for presentation.

5.2 UI-to-Database Interaction



Correct presentation path: browser action → Angular service → Spring validation → JPA persistence → refreshed UI state.

Figure 18: UI click-to-database flow

This sequence-style interaction view emphasizes the complete request lifecycle from a browser click through backend validation and persistence to the refreshed UI result.

6. Software Architecture & Traceability

Layered Runtime and Deployment View

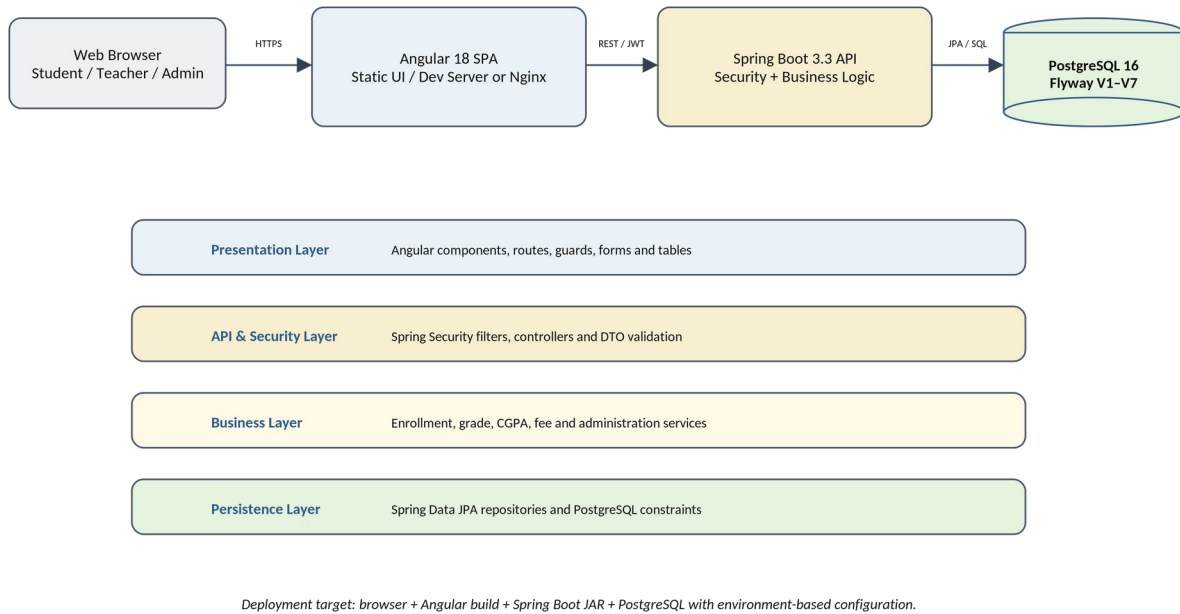


Figure 19: Layered runtime and deployment view

Architecture decision: Use a straightforward Angular SPA → Spring Boot REST API → PostgreSQL design. This matches the current project, supports easy deployment and avoids unnecessary infrastructure.

6.1 Requirement Traceability Matrix

Requirement	Use Case / Diagram	Implementation Area	Status
FR-01/02	UC Authentication; Figure 3/8	Login UI, AuthController, SecurityConfig, users	Implemented
FR-03/04	UC Registration; Figure 4/9/15	Course UI, EnrollmentController/Service, enrollments	Limited
FR-05/06	Figure 9/15	Required SemesterRegistration + CourseOffering transaction	Target
FR-07/08	UC Teacher; Figure 6/10	Teacher UI, GradeController/Service, grades	Implemented
FR-09/10	Figure 11	Grade/CGPA logic; required publication state	Limited
FR-11/12	Figure 12	Fee services; required payment/allocation workflow	Limited / Target
FR-13/14	UC Admin; Figure 7	Admin UI/API, statistics and master-data tables	Limited

7. Preliminary Test Plan

ID	Area	Pass Condition
TC-01	Authentication	Valid credentials issue JWT; invalid credentials reveal no sensitive detail.
TC-02	Role isolation	Student, Teacher and Admin cannot access another role namespace.
TC-03	Credit rule	Exactly 12 credits accepted; other totals rejected without partial records.
TC-04	Duplicate enrollment	Second student-course-semester association is rejected.
TC-05	Seat control	Full-course request fails and seat count remains correct under concurrency.
TC-06	Marks range	Values outside allowed midterm/final range are rejected.
TC-07	Grade / CGPA	Known marks and credits produce the expected grade points and CGPA.
TC-08	Payment / dues	Verified payment changes due by the allocated amount only.
TC-09	Migration / build	Fresh Flyway V1-V7 migration, Maven build and Angular build succeed.

8. Implementation Progress & Challenges

Status	Capability	Evidence / Remaining Work
Completed	Foundation, JWT/RBAC and core entities	Angular/Spring/PostgreSQL structure, authentication and role-based routes exist.
Completed	Basic Student and Teacher workflows	Dashboard/profile, course list, course-wise enrollment/drop, grade entry and CGPA view exist.
In Progress	Admin and finance experience	Core CRUD, fees and statistics exist but remain incomplete.
Not Yet Correct	Exact-12 aggregate registration	Current code validates ≤ 12 and saves course-by-course.
Not Yet Correct	Semester-specific offering and last-seat safety	Seats and Teacher remain on Course; transaction/locking correction required.
Not Yet Complete	Publication, verified payment and gap automation	Explicit result state, payment allocation and gap detection remain required.

Highest-priority challenges: align the Angular enrollment DTO with semesterId, introduce CourseOffering, remove duplicate seat mutation, derive retake/gap status from history and create explicit result/payment lifecycles.

9. Timeline & Demonstration Plan

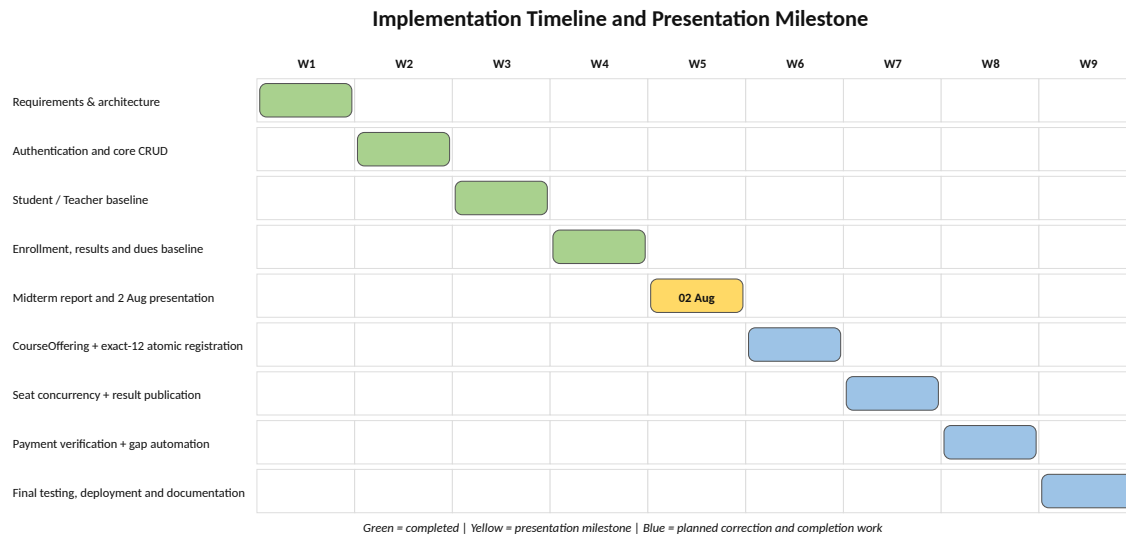


Figure 20: Timeline and 2 August presentation milestone

9.1 Presentation Demonstration Sequence

Time	Action	Expected Evidence
0:00–0:45	Explain problem, actors and Angular → Spring Boot → PostgreSQL architecture.	Figures 1, 17 and 19.
0:45–1:30	Login as Student and open the role dashboard.	JWT session and protected route.
1:30–3:15	Open course catalogue and perform an enrollment action.	Real API request and saved/updated enrollment view.
3:15–4:30	Login as Teacher and show roster/mark entry.	Assigned-course ownership and grade validation.
4:30–5:30	Show CGPA or dues loaded from backend data.	Second database-backed feature.
5:30–6:30	Explain current limitations and next correction plan honestly.	Progress table, DFD Level 2 and Gantt chart.